

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	Shahanaj.c	Reg. No.:	192372292
Section / Batch:	CSE	Date:	16-07-2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. Real-Time Search Index Compression A search indexing system reduces its input size by half at each recursive step. At every level, it performs additional indexing work proportional to the logarithm of the current input size. Apply the substitution method to solve the recurrence $T(n) = T(n/2) + \log_2 n$. Expand the recurrence step-by-step and derive the final asymptotic time complexity. Analyze how the recursion depth influences performance. Interpret the efficiency using asymptotic notation.

Q2. Intelligent Decision Support Module An optimized decision-support algorithm reduces the problem size by half at each step and performs only constant work for evaluation and branching. Apply the substitution method to solve the recurrence $T(n) = T(n/2) + 1$. Show the recursive expansion clearly and derive the final asymptotic complexity. Analyze how this improves efficiency for large inputs. Interpret the result using asymptotic notation.

Code of Conduct

I shahanaj-(192372292)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: shahanaj_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1.Real-Time Search Index Compression

Given recurrence:

$$T(n) = T\left(\frac{n}{2}\right) + \log n$$

Assume the base condition is:

$$T(1) = \Theta(1)$$

Step-by-Step Expansion

Substitute $T(n/2)$:

$$T(n) = \left[T\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right) \right] + \log n$$

$$T(n) = T\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right) + \log n$$

After the next substitution:

$$T(n) = T\left(\frac{n}{8}\right) + \log\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right) + \log n$$

After k substitutions:

$$T(n) = T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} \log\left(\frac{n}{2^i}\right)$$

Using the logarithm rule:

$$\log\left(\frac{n}{2^i}\right) = \log n - i \log 2$$

Assuming logarithms are base 2:

$$\log\left(\frac{n}{2^i}\right) = \log n - i$$

Therefore:

$$T(n) = T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} (\log n - i)$$

Finding the Recursion Depth

The recursion stops when the input size becomes 1:

$$\begin{aligned} \frac{n}{2^k} &= 1 \\ n &= 2^k \\ k &= \log_2 n \end{aligned}$$

Thus, the recursion depth is:

$$\Theta(\log n)$$

Let $m = \log_2 n$. Substituting $k = m$:

$$T(n) = T(1) + \sum_{i=0}^{m-1} (m - i)$$

The summation becomes:

$$m + (m - 1) + (m - 2) + \cdots + 1$$

Using the sum of the first m natural numbers:

$$T(n) = T(1) + \frac{m(m + 1)}{2}$$

Substituting $m = \log n$:

$$T(n) = \Theta(1) + \frac{\log n(\log n + 1)}{2}$$

$$T(n) = \frac{(\log n)^2 + \log n}{2} + \Theta(1)$$

Therefore, the dominant term is:

$$T(n) = \Theta((\log n)^2)$$

Influence of Recursion Depth

The input is divided by 2 at every recursive step:

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, 1$$

Hence, only $\log_2 n$ recursive calls are required. However, the indexing work performed at each level decreases as:
 $\log n, \log n - 1, \log n - 2, \dots, 1$

The total work is therefore the sum of approximately $\log n$ decreasing logarithmic terms, producing a squared logarithmic complexity.

Efficiency Interpretation

$$T(n) = O((\log n)^2)$$

$$T(n) = \Omega((\log n)^2)$$

$$T(n) = \Theta((\log n)^2)$$

This is highly efficient because the running time grows very slowly even for large input sizes. For example, when $n = 1024$:

$$\log_2 1024 = 10$$

So the total additional work is approximately:

$$10 + 9 + 8 + \dots + 1 = 55$$

Thus, the search indexing system has logarithmic recursion depth and overall polylogarithmic time complexity:

$$\Theta(\log^2 n)$$

2. Intelligent Decision Support Module

Given recurrence:

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Assume the base condition:

$$T(1) = \Theta(1)$$

Step-by-Step Recursive Expansion

First substitution:

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Expand $T(n/2)$:

$$T(n) = \left[T\left(\frac{n}{4}\right) + 1 \right] + 1$$

$$T(n) = T\left(\frac{n}{4}\right) + 2$$

Expand again:

$$T(n) = T\left(\frac{n}{8}\right) + 3$$

After k substitutions:

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

Finding the Recursion Depth

The recursion stops when the input size becomes 1:

$$\begin{aligned}\frac{n}{2^k} &= 1 \\ n &= 2^k\end{aligned}$$

Taking logarithm on both sides:

$$k = \log_2 n$$

Therefore, the recursion depth is:

$$\Theta(\log n)$$

Substitute $k = \log_2 n$ into the expanded recurrence:

$$T(n) = T(1) + \log_2 n$$

Since $T(1) = \Theta(1)$:

$$T(n) = \Theta(1) + \log_2 n$$

Therefore:

$$\boxed{T(n) = \Theta(\log n)}$$

Efficiency for Large Inputs

The algorithm reduces the input size by half at every step:

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, 1$$

Since only constant work is performed at each level, the total running time depends mainly on the number of recursive levels.

For example, when:

$$\begin{aligned}n &= 1024 \\ \log_2 1024 &= 10\end{aligned}$$

Therefore, only about 10 recursive steps are required.

Even if the input size doubles from 1024 to 2048, the number of steps increases only from 10 to 11. This makes the algorithm highly efficient for large inputs.

Asymptotic Interpretation

The upper bound is:

$$T(n) = O(\log n)$$

The lower bound is:

$$T(n) = \Omega(\log n)$$

The tight bound is:

$$\boxed{T(n) = \Theta(\log n)}$$

Thus, the optimized decision-support algorithm has **logarithmic time complexity**. It performs efficiently because the problem size decreases rapidly while only constant evaluation and branching work is done at each recursive level.